are running with the same base configuration and packages, where configuration files need to be checked-out to a version-controlled repository. Only planned changes are allowed and the previous configuration state is restored for unplanned changes. Additionally, centralised user and policy management, along with automated configuration recovery during disaster conditions are required. In such a case, building a team of 10-20 administrators would not be a recommended approach. Rather, using a centralised configuration tool to automate the administration tasks would be a better option to follow.

Along with commercial tools like BladeLogic and OpsWare, there are a couple of open source systems automation and configuration management tools available like Bcfg2, Cfengine and Puppet. Cfengine has been an administrator's favourite configuration management framework since the past few years and is widely being used by many companies. Puppet turns out to be a next-generation configuration management tool to overcome many of Cfengine's weaknesses.

Puppet is written in Ruby and is released under the GPL. It supports a number of operating systems like CentOS, Debian, FreeBSD, Gentoo, OpenBSD, Solaris, SuSE Linux, Ubuntu, etc. Puppet is being used by many organisations including Google, which uses it to manage all Mac desktops, laptops and Linux clients. A list of other Puppet users can be fetched from *reductivelabs.com/trac/puppet/wiki/WhosUsingPuppet*

Puppet installation

Puppet installation is fairly easy and is, in fact, a matter of seconds. Puppet runs in client-server configuration, where the client polls the server at port 8140 every 30 minutes to check for the new instructions or to match the configuration files. The client also listens to a port to have push-updates from the server. In Puppet terminology, a client is called a Puppet node and a server is called a Puppet master. Figure 1 shows the set-up.

The following few steps demonstrate the installation steps for the CentOS operating system—a similar approach can be followed for other supported systems:

On the server side:

1. Define the hostname for server as *puppet.domain.com*
2. Puppet can be installed using *yum*, but packages are not part of the default CentOS repositories or installation DVD. Even though it is available at DAG's repository, the versions are outdated. The best repository for Puppet is EPEL (Extra Packages for Enterprise Linux—see *fedoraproject.org/wiki/EPEL*). Puppet RPMs can either be directly downloaded and installed, or the *yum* repository can be configured to do the job. To use the EPEL repository, run the following command as a root user:

```
rpm -Uvh http://download.fedora.redhat.com/pub/epel/5/i386/epel-release-5-3.noarch.rpm
```

3. Now install the Puppet server by issuing the following command:

```
yum install puppet-server
```

4. Install *ruby-rdoc* to enable Puppet command line help:

```
yum install ruby-rdoc
```

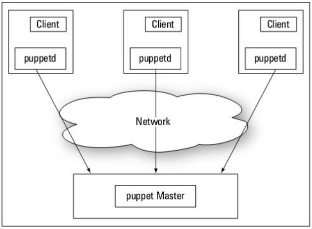

Figure 1: A typical Puppet set-up

5. Now, create a sample manifest file to start the Puppet server. This is just a test manifest and more complex manifests can be created using this tool, which will be demonstrated later. Put the following contents into the file using Vim or any other text editor. The purpose here is to create */tmp/testfile* on a node (puppet client) if it doesn't exist:

```
class test_class {
  file { "/tmp/testfile":
    ensure => present,
    mode   => 644,
    owner  => root,
    group  => root
  }
}
node puppetclient {
  include test_class
}
```

In the above content, the upper section defines a class named *test_class* that ensures that */tmp/testfile* with the defined permission is present on the client where the class will be included. In the lower section, client *puppetclient* includes the *test_class* and Puppet will create the file with the set permission on *puppetclient* if it doesn't already exist. Once done, start the Puppet server using the following command:

```
service puppetmaster start
```

6. The Puppet server is now installed and configured to listen to incoming connections from agents. Default installation comes with Webrick, which is not a good Web server to handle loads from a higher number of Puppet agents. Apache and Mongrel can solve this problem. Refer to the Puppet wiki for instructions on configuring Puppet with Mongrel.

On the client side:

1. Define the hostname for the server as *puppetclient.domain.com*
2. Configure the EPEL repository using the following command again: